

Project

Deadline: 3rd May 2026 11:59PM

Instructions:

- All the submissions must be made on Google classroom.
- You must submit files in a ZIP Folder named (24I-XXXX_24I-XXXX_24I-XXXX.zip).
The naming convention must be followed strictly.
- A **group of up to 3 members** is allowed; **cross section groups are not permitted.** •
Students are solely responsible for checking the final zip files for issues like corrupt files, viruses in the file.
- Be prepared for viva or any follow up evaluation after Project submission. •
You are allowed to use AI tools only for frontend development.
- Zero marks will be awarded to the students involved in plagiarism.
- Late submission is not allowed.

Smart Disaster Response MIS with Secure Transactions & Role-Based Control

A country frequently faces natural disasters such as floods, earthquakes, and urban fires. During such events, thousands of emergency reports are generated in real time through mobile applications, helplines, and monitoring systems. To efficiently manage crisis situations, the government plans to develop a **Smart Disaster Response Management Information System (MIS)**. This system will be used by multiple stakeholders including emergency operators, field officers, warehouse managers, finance officers, and administrators. The system must support **real time coordination, high-volume transactions, secure data handling, analytical reporting, database-level automation and performance optimization mechanisms**, along through a complete frontend interface, making it a **full-fledged enterprise-level system**.

System Requirements

1. Emergency Reporting

Citizens can report emergencies by providing:

- Location of incident
- Type of disaster
- Severity level
- Time of report

The system should:

- Handle continuous inflow of reports (high-frequency inserts)
- Allow real-time status tracking of each report
- Support filtering and prioritization of incidents

2. Rescue Team Management

The system maintains records of rescue teams, including:

- Team type (medical, fire, rescue)
- Current location
- Availability status

Additional requirements:

- Dynamic assignment based on proximity and severity
- Real-time status updates (Available → Assigned → Busy → Completed) •
- Historical tracking of team activities

3. Resource Management

The system tracks resources such as:

- Food supplies
- Water
- Medicines
- Shelter equipment

Enhancements:

- Warehouse-wise inventory tracking
- Resource threshold alerts (low stock warnings)
- Multi-step resource allocation workflows
- Tracking of dispatched vs consumed resources

4. Hospital Coordination

Hospitals provide real-time updates about:

- Available beds
- Admitted patients
- Critical cases

Additional features:

- Automated patient assignment based on hospital capacity
- Emergency escalation handling
- Hospital load balancing

5. Financial Management

The system records:

- Donations from individuals and organizations
- Expenses related to disaster response
- Resource procurement and distribution costs

Enhancements:

- Transaction categorization (donation, expense, procurement) •
- Budget tracking per disaster event
- Financial audit trails

6. High-Volume Transaction Processing

The system must handle a large number of concurrent operations such

- as:
- Emergency report submissions
 - Resource allocations and distributions
 - Rescue team assignments
 - Financial transactions

Requirements:

- Concurrency control mechanisms
- Prevention of data inconsistency
- Handling of race conditions and conflicts

7. Secure Transaction Requirements

Certain operations involve multiple dependent steps and must be treated as a **single logical unit (transaction)**.

Examples:

- Allocating resources and updating warehouse inventory
- Assigning rescue teams and updating availability
- Recording financial transactions

The system must ensure:

- Atomicity, Consistency, Isolation, Durability (ACID properties)
- Rollback mechanisms in case of failure
- Proper transaction logging

8. Role-Based Access Control (RBAC)

User roles include:

- Administrator
- Emergency Operator
- Field Officer
- Warehouse Manager
- Finance Officer

Requirements:

- Fine-grained access control
- Role-based permissions for CRUD operations
- Restricted data visibility per role

9. Approval-Based Workflow

Critical actions must follow an approval process:

Examples:

- Resource distribution requests
- Rescue deployment requests
- Financial approvals

System behavior:

- Requests stored in *Pending* state
- Approval/Rejection by authorized roles
- Execution only after approval

- Maintain approval history

10.Data Security & Privacy

The system must ensure:

- Secure authentication (login system)
- Encrypted password storage
- Protection of sensitive financial and user data
- Authorization checks before every critical operation

11.MIS Reporting & Analytics

The system should generate:

- Incident statistics by location/severity
- Resource utilization reports
- Response time analytics
- Financial summaries
- Approval workflow reports

Advanced expectations:

- Dashboard-based visualization
- Interactive filtering and drill-down analysis

12.Audit & Monitoring

The system should maintain logs of:

- User actions
- Data modifications
- Approval decisions

Requirements:

- Full traceability
- Timestamped logs
- Support for auditing and compliance checks

13.Advanced Database Behavior & Automation

The system must incorporate **database-level intelligent mechanisms** to automate operations and improve performance:

Triggers (Event-Driven Automation)

The database should automatically respond to critical events such as:

- Updating resource stock after allocation or dispatch
- Changing rescue team status upon assignment or completion
- Logging financial transactions into audit tables
- Enforcing business rules (e.g., preventing negative inventory)

These triggers must ensure:

- Consistency without relying solely on application logic
- Automatic propagation of changes across related tables
- Reduction of manual intervention in critical workflows

Views (Logical Data Abstraction & Access Control)

The system must define multiple database views to:

- Provide **role-specific data visibility** (e.g., finance vs field officer)
- Simplify complex joins for reporting and dashboards
- Restrict access to sensitive attributes

Additionally, the system should require:

- Comparative evaluation of **query performance using views vs direct table queries** •

Identification of cases where views provide:

- Faster response (optimized joins, pre-structured queries)
- Better security and abstraction
- Measurement of **query latency differences** when using views versus base tables

14. Performance Optimization & Indexing

Given the high-volume nature of the system, database performance must be critically analyzed:

Custom Indexing

Students must:

- Create indexes on frequently queried attributes such as:
 - Incident location
 - Disaster type
 - Resource type
 - Transaction timestamps
- Use both **single-column and composite indexes**

Query Performance Analysis

The system must include:

- Comparative analysis of:
 - Queries executed **with indexing vs without indexing**
- Measurement of:
 - Query execution time
 - Response latency
- Identification of:
 - Scenarios where indexing improves performance
 - Cases where indexing may introduce overhead (e.g., inserts/updates)

15. Frontend Requirements

The system must include a **complete user interface**:

- Web-based dashboard for different roles
- Role-specific views and functionalities
- Forms for:
 - Emergency reporting
 - Resource requests
 - Financial entries

- Interactive dashboards (charts, tables)
- Real-time updates (or near real-time simulation)

16.Database Design Requirements

Students must design and submit:

ERD (Entity Relationship Diagram)

- Identify all entities
- Show relationships
- Include primary and foreign keys

Relational Schema

- Convert ERD into tables
- Clearly define:
 - Attributes
 - Primary keys
 - Foreign keys

17.Design Rationale

Students must provide justification for:

- Database design decisions
- Choice of entities and relationships
- Transaction handling approach
- RBAC implementation strategy
- Indexing and performance considerations

18.Full-Fledged System Expectations

The project should be treated as a **complete system**, not just a database:

Students must demonstrate:

- Integration of frontend + backend + database
- Realistic workflows (end-to-end functionality)
- Handling of concurrent operations
- Error handling and validation
- Modular and scalable design

19.Design Rationale

Students must now also justify:

- Use of triggers and their impact on automation
- Use of views for abstraction and security
- Performance comparison (views vs tables)
- Indexing strategy and observed latency improvements

Trade-offs between read and write performance

Final Deliverables

Students must submit:

1. ERD Diagram
2. Relational Schema
3. Normalization Steps
4. SQL Implementation (DDL + DML + Queries)
5. Transaction Handling Demonstration
6. Trigger Implementation & Use Cases
7. View Definitions & Performance Comparison (Latency Analysis)
8. Indexing Strategy & Query Performance Report
9. Frontend Interface (screens or working app)
10. Design Rationale Document
11. MIS Reports / Dashboards
12. System Demonstration (optional but recommended)